GPU-Resident AES-CTR Encryption of Independent Data Pages: Computational Gains and Transfer Limits

Marko Mihajlovic[1]

Abstract: Accelerating AES-CTR with a GPU is useful only when the gains from parallel computation exceed host–device data-movement costs. This study investigates that condition using validated AES-128 CUDA implementations, a native OpenSSL EVP AES-NI baseline, and a batch of independent 1 MiB data pages motivated by columnar storage encryption. Five repeated Tesla T4 sessions show that, for 32 independently encrypted pages (32 MiB), GPU-resident AES-CTR computation achieves a 2.251× speedup relative to the prepared native CPU transformation, whereas the transfer-inclusive prepared GPU pipeline achieves only 0.913×. The corresponding median times are 3.714 ms for the GPU resident kernel, 8.362 ms for OpenSSL transformation, and 9.153 ms for the GPU pipeline. Page-aware GPU execution has approximately the same pipeline throughput as a continuous 32 MiB CTR stream (a 1.007× page-to-continuous ratio), indicating little added cost for independent counter domains in this implementation. In a separate same-runtime comparison, the apparent 32 MiB GPU advantage changes from 5.586× against PyCryptodome to 0.942× against native OpenSSL. The practical opportunity is therefore GPU-resident encryption as a stage within an already device-resident workflow, not general-purpose offloading of host-resident data. That application-level opportunity remains a conditional implication rather than a measured end-to-end application speedup.

Keywords: AES-CTR, GPU-resident encryption, independent data pages, CUDA, AES-NI, OpenSSL, data movement, heterogeneous processing.

1 Introduction

The Advanced Encryption Standard (AES) is widely used to protect stored and transmitted data [1]. Counter (CTR) mode exposes independent block-encryption operations and is amenable to parallel execution [2]. Nevertheless, a GPU is not automatically the fastest place to perform encryption: host processors provide dedicated AES instructions, and moving plaintext and ciphertext across the CPU–GPU boundary consumes time. An application using an accelerator must therefore distinguish the cost of AES computation from the cost of placing data where that computation occurs.

This distinction matters for storage and analytics workloads consisting of many

[1] Singidunum University, Belgrade, Serbia; ORCID: 0009-0006-7979-0341

independent data units. The Apache Parquet encryption specification provides a concrete motivation: its `AES_GCM_CTR_V1` mode uses CTR encryption for data pages and GCM protection for other encrypted modules. Rather than claim support for a complete Parquet file, we study a controlled, Parquet-inspired workload of independent 1 MiB pages with a distinct 96-bit nonce and a 32-bit counter initialized to one for each logical page. The study does not implement file framing, authenticated metadata, key management, compression, or Parquet interoperability. CTR encryption alone does not provide authentication.

The central question is not simply whether a GPU can execute AES quickly, but *where a measured advantage exists and what must change in data placement for an application to exploit it*. A GPU-resident input can be encrypted without round-trip payload movement. A CPU-resident input that must return to host memory after GPU encryption requires two transfers. These are distinct workflows and must not be described by a single generic "GPU speedup."

The study makes three bounded contributions. First, it establishes a same-runtime comparison with native OpenSSL EVP AES-NI, correcting the interpretation of earlier comparisons that used PyCryptodome as their host reference. Second, it quantifies how a GPU compute-only advantage for 32 MiB CTR is lost at a prepared transfer-inclusive boundary. Third, it validates independent nonce/counter domains per page and shows that their GPU processing cost is close to the cost of a continuous stream of equal length. The contribution is a measured execution-placement distinction and a reproducible page-aware implementation, not a new AES primitive or a demonstrated complete storage-engine speedup.

Previously recorded measurements on GTX 1650 Ti, T600 and Tesla T4 are retained as historical, implementation-specific evidence. New same-runtime T4 CPU/GPU and page-batching sessions are presented separately; their observations are not pooled with the earlier data. Section 3 defines the measurement boundaries, Section 4 presents the positive and negative results together, and Sections 5 and 6 discuss the practical conditions and remaining work.

2 Related Work and Research Gap

Early contributions by Manavski [3] and Harrison and Waldron [4] established the feasibility of AES execution on programmable GPU architectures. Subsequent work investigated how parallel processing, memory organization, and architecture-specific optimization affect encryption throughput. The available parallelism depends partly on the encryption mode: ECB permits independent block transformations, while CTR generates keystream from distinct counter values [2]. Both expose independent work for GPU threads when buffers are sufficiently large. ECB is unsuitable for protecting general structured data and serves here only as a

controlled performance reference.

Iwai et al. [5] examined thread granularity, memory organization, and the interaction between computation and data transfers, including overlapping transfers with computation. Their CUDA implementation achieved approximately 35 Gbit/s for device-side AES processing and approximately 22.5 Gbit/s for transfer-aware execution. These results demonstrate that the execution boundary and transfer strategy affect the reported encryption rate. Wang and Chu [6] investigated lookup-table-based AES implementations and GPU execution strategies. Tezcan [7] examined shared-memory optimization and bank-conflict avoidance, reporting AES-128 encryption throughput of 878.6 Gbit/s on an NVIDIA RTX 2070 Super.

More recent work has explored bit-sliced cipher representations. A 2024 study [8] reported high-performance AES-CTR and AES-ECB execution on desktop and embedded NVIDIA platforms. Together, these studies demonstrate the computational capabilities of optimized GPU implementations. Their reported rates must nevertheless be interpreted within their respective measurement boundaries: device-side encryption throughput does not directly specify the rate at which an application can receive, encrypt, and deliver data. It also does not establish whether offloading improves performance relative to the host processor available in a particular system.

That comparison depends on the CPU implementation. Dedicated instructions such as AES-NI reduce the cost of AES transformations [9] and provide a hardware-assisted baseline for evaluating GPU offloading. Lee et al. [10] demonstrated that conclusions about heterogeneous acceleration depend strongly on the optimization level of the implementations being evaluated. Their findings support using an appropriately accelerated CPU reference when deciding whether GPU offloading offers an additional processing benefit. Accordingly, comparing GPU encryption with conventional unaccelerated software cannot establish an advantage over the host's hardware-assisted AES path. The reference must represent the execution path against which an offloading decision will actually be made, and the comparison remains specific to the implementations evaluated.

Data location further determines which costs enter that decision. Gregg and Hazelwood [11] showed that CPU–GPU performance evaluation requires explicit consideration of data movement. An advantage for device-resident input may shrink when both input and output cross the host–device boundary. CUDA guidance recommends minimizing unnecessary transfers, using appropriate synchronization, and considering overlap between communication and computation [12, 13]. The transfer-aware AES work of Iwai et al. [5] demonstrates that this concern is already established in GPU cryptographic processing.

For communication infrastructure, these findings connect encryption per-

formance to the surrounding processing path. A host-managed interface may receive data, organize buffers, invoke encryption, and forward the output. The resulting costs differ between contiguous buffers, aggregated independent messages, and individual latency-sensitive packets. Device-resident measurements and transfer-inclusive measurements therefore answer different questions. The present study distinguishes resident execution from a synchronous prepared-buffer pipeline containing host-to-device and device-to-host transfers. This pipeline measures a broader operation than the kernel alone, but excludes complete network input/output and application processing; it also does not evaluate transfer–computation overlap.

Telecommunications security adds requirements beyond encryption throughput. AES-GCM combines counter-mode encryption with GHASH authentication [14]; an improvement in AES-CTR can therefore concern one component of an authenticated-encryption pipeline without establishing an equivalent improvement in IPsec or TLS processing. Practical implementations must also account for nonce management, packet preparation, synchronization, and output delivery. These requirements distinguish a bulk encryption benchmark from a complete security service, even when the underlying cipher operations are parallelizable.

Earlier research also examined GPU-assisted encryption in communication protocols. Schönberger and Fuß [15] investigated GPU precomputation of AES counter-mode keystream for an IPsec-oriented GCM implementation, with subsequent encryption and authentication performed on the CPU. More recently, Lee et al. [16] developed a parallel GPU implementation of GCM, including GHASH processing, and reported throughput exceeding 400 Gbit/s on an RTX 4090. Thus, GPU-assisted telecommunications encryption and authenticated-encryption acceleration are established research directions. The present study instead characterizes reproducible operating conditions for bulk AES-128 processing, distinguishing device-resident execution from synchronous transfer-inclusive processing against hardware-assisted AES-NI. It neither introduces a new AES algorithm nor benchmarks complete AES-GCM or IPsec processing.

Workload aggregation introduces a further throughput–latency trade-off. Combining small messages may increase the work available to the GPU, while waiting for a batch adds queueing delay. Bulk data protection, traffic aggregation, heterogeneous communication gateways, and data-processing services are consequently possible application contexts, but their suitability depends on host acceleration, data placement, and operational requirements. Peak kernel rates alone cannot determine that suitability. The present measurements characterize contiguous bulk buffers and do not demonstrate packet batching or complete telecommunications protocol performance.

Assessing whether an observed advantage is repeatable also requires control of experimental variability. Mytkowicz et al. [17] showed that experimental

conditions can introduce systematic bias and alter performance results. Kalibera and Jones [18] emphasized the importance of accounting for variability across multiple levels of experimental replication, while Hoefler and Belli [19] discussed scientifically defensible performance evaluation of parallel computing systems. These contributions provide a methodological basis for determining whether an acceleration is repeatable under the conditions tested. These studies motivate reporting repeated observations and their execution context rather than relying on the highest throughput measurement. For cryptographic workloads, functional correctness must additionally be verified before timings are accepted as evidence of encryption performance.

The present evaluation applies these principles through cryptographic validation, paired workloads, repeated study sessions, documented timing boundaries, and preservation of raw observations and experimental provenance. Session-level evidence distinguishes isolated favorable measurements from advantages observed consistently across executions, while retaining outcomes that favor the CPU. Detailed procedures and aggregation equations are given in Section 3.

The literature thus establishes GPU AES feasibility, optimization techniques, and the need to account for transfers, CPU acceleration, and measurement variability. The question addressed here is which operating conditions produce a repeatable bulk-processing benefit when the host already supports hardware-assisted encryption. High kernel throughput alone cannot answer this question, and an unfavorable transfer-inclusive result on one system cannot settle it for other platforms or workloads.

To examine these conditions, the study compares AES-128 ECB and CTR workloads of different sizes across three heterogeneous configurations under a common validation and measurement protocol. The comparison uses hardware-assisted CPU AES-NI as its reference and retains session-level measurements for assessing consistency across repeated executions. Its contribution is the identification of experimentally supported operating conditions in which GPU-assisted bulk AES processing provides an advantage, together with evidence of constraints under which that advantage disappears or becomes inconsistent. This supports assessment of GPU-assisted bulk encryption and workload placement within the tested execution boundaries. Extending those findings to authenticated encryption and network processing requires separate application-level measurements.

3 Methodology

The evaluation compares AES-128 encryption using hardware-assisted CPU AES-NI and a validated CUDA implementation. CTR is the primary workload; ECB provides a controlled reference for mode-dependent implementation behavior.

Both modes process contiguous buffers under the same validation and measurement protocol. Three complete hardware–software configurations are evaluated: two Windows systems and a Linux-based cloud allocation. Table 1 lists the recorded hardware and software. The exact model of the cloud host CPU was unavailable in the archived environment records.

Table 1
Hardware and software configurations used for AES acceleration experiments.

ID	Hardware and environment	Software stack
GTX	Intel Core i5-10300H; GeForce GTX 1650 Ti, 4 GB; Windows 10	Driver 610.62; CUDA runtime 12.9; CuPy 13.6.0; PyCryptodome 3.23.0
T600	Intel Core i7-11850H; T600 GPU, 4 GB; Windows 11	Driver 581.95; CUDA runtime 12.9; CuPy 14.2.0; PyCryptodome 3.24.1
T4	Intel Xeon at 2.00 GHz, two exposed logical processors; Tesla T4; Linux/Google Colab	Driver 580.82.07; CUDA runtime 13.0; CuPy 14.0.1; PyCryptodome 4.0.0

3.1 Independent ARM64 CPU Control

An additional CPU-only study was conducted on an Oracle Cloud ARM64 instance with two available vCPUs (Arm Neoverse-N1) and no GPU. It is reported as an architecture and implementation control, not as a fourth CPU–GPU pair. Five single-worker sessions measured AES-128-CTR using PyCryptodome, native OpenSSL 3 EVP with the default CPU capabilities, and an OpenSSL run with `OPENSSL_armcap=0`. A separate five-session, two-worker experiment recorded aggregate operation-envelope throughput. All comparisons in this subsection are limited to the Oracle host and their explicitly named measurement intervals. In particular, `OPENSSL_armcap=0` disables multiple Arm acceleration capabilities, so its performance difference is not an isolated measurement of a single AES instruction. Native EVP transform-only timing excludes allocation, Python interoperation, and output conversion; operation-envelope timing includes additional host-side costs. The two-worker results are aggregate throughput, not a twofold reduction in a single request's latency.

The CPU reference uses PyCryptodome with hardware-assisted AES-NI explicitly enabled [9, 20]. Native backend initialization was checked for both modes; executed processor instructions were not traced directly. The GPU path uses the validated `cuda_baseline` kernel through CuPy's custom kernel interface [21]. Each CUDA thread processes one 16-byte AES block, with 128 threads per CUDA block. These paths are selected from a broader campaign that evaluated additional implementations. The analysis retains their original execution context,

including interleaved jobs excluded from the selected results.

Cryptographic validation precedes acceptance of timing measurements. Known-answer tests and consistency checks against CPU reference operations use the AES specification and mode definitions [1, 2]. The archived records document successful CUDA execution and agreement with expected outputs. Executions failing the required checks are excluded from valid performance results. Experimental configuration, measurement-record completeness, and source-code identity are also verified.

The archived harness validates each measured output against the CPU reference outside the timed call, in addition to the initial validation gate. An output mismatch stops performance export. For each size, mode, and repetition, a single synthetic message is replayed across the paired backends. Plaintext and a 16-byte key are generated from the seeded random generator. The initial CTR block consists of eight generated bytes followed by eight zero bytes. The CPU CTR call uses an empty nonce and interprets the full initial counter as a big-endian integer.

Pair identifiers record size, mode, and repetition. A SHA-256 digest of the concatenated key, counter, and plaintext records the common input material. Raw records also retain the session identifier, seed, backend, mode, measurement scope, job order, global execution order, and validation flag. These fields allow the selected CPU and GPU observations to be matched without pooling observations from different sessions. Previous output buffers are released outside the next timed call.

The retained dataset is checked against the archived file hashes. For each platform, mode, size, and repetition, the selected observations contain one CPU transformation, one resident GPU measurement, and one pipeline measurement with matching input digests. Each configuration group contains all twenty recorded repetitions. Throughput is independently checked against the corresponding byte count and recorded time before session-level aggregation. Resident records specify one kernel iteration, and the GPU records identify pinned host memory. The verification reconstructs the aggregates from raw records rather than relying solely on the saved summary tables.

The experiment uses five input sizes:

$$N \in \{1\ \text{KiB},\ 16\ \text{KiB},\ 1\ \text{MiB},\ 16\ \text{MiB},\ 32\ \text{MiB}\}. \tag{1}$$

Corresponding CPU and GPU executions use deterministic input generation and matching plaintext, encryption keys, and mode-specific parameters. Each platform has five study sessions with random seeds 2003–2007. For each measured configuration, five warm-up executions precede twenty recorded repetitions; warm-ups are excluded from the timing records. Each session contains thirty selected

configuration groups, comprising two modes, five sizes, and three execution boundaries. With twenty observations per group, each session contributes 600 observations, giving

$$3 \times 5 \times 30 \times 20 = 9\,000 \tag{2}$$

raw timing observations across the three systems. Workload sizes execute in ascending order, while mode and backend/measurement-scope jobs are shuffled using the session seed. Benchmark jobs run sequentially. The selected observations were extracted from the existing campaign, without rerunning or reordering the benchmark schedule.

Three timing boundaries are retained:

1. **CPU AES-NI transformation.** The interval measures the host encryption call, including output allocation performed by that call. Cipher construction and key setup are excluded.
2. **GPU resident execution.** CUDA events measure a single kernel launch with input already in device memory. Host-to-device and device-to-host transfers are excluded.
3. **Prepared-buffer GPU pipeline.** The interval measures the synchronous host-driven sequence of host-to-device transfer, CUDA kernel execution, device-to-host transfer, and synchronization required to complete these operations.

CPU execution and the GPU pipeline are timed with Python's monotonic high-resolution `time.perf_counter_ns()` interface. Resident execution uses CUDA event timing with the required synchronization, following CUDA and CuPy measurement guidance [12, 22]. Thus, the CPU and pipeline intervals are measured from the host, while the resident interval measures device-side execution.

The prepared-buffer pipeline uses pinned host memory and preallocated device buffers, prepared before the timed interval. Its processing stages are represented as

$$T_{\text{pipeline}} = T_{\text{H2D}} + T_{\text{kernel}} + T_{\text{D2H}} + T_{\text{overhead}}, \tag{3}$$

where H2D and D2H denote host-to-device and device-to-host transfers, and the final term covers additional costs within the measured boundary. This expression describes the stages of the synchronous operation; it is not a decomposition calculated by adding separately aggregated component measurements. Recorded transfer-component intervals include synchronization effects and are not isolated measurements of physical PCIe transmission time.

GPU compilation, memory allocation and pinning, initial host-side input staging, key preparation, and application-level input/output are excluded from the pipeline interval. Transfers and computation are not overlapped using multiple asynchronous CUDA streams. Resident execution and prepared-buffer processing are measured separately, with the latter including the movement needed to process a host-resident buffer and return its output. No network input/output, packet aggregation, or authenticated-encryption operation is included in these measurements.

Elapsed processing time, throughput, and host-relative GPU acceleration are summarized at session level. For an input of N bytes and elapsed time $T_{s,r}$ in seconds at repetition r of session s, throughput in decimal MB/s is

$$R_{s,r} = \frac{N}{10^6 T_{s,r}}, \tag{4}$$

Input sizes use binary KiB and MiB units. Time and throughput medians are calculated independently within each session, then aggregated across the five session estimates:

$$\widetilde{T} = \text{median}_s \left(\text{median}_r \, T_{s,r} \right), \qquad \widetilde{R} = \text{median}_s \left(\text{median}_r \, R_{s,r} \right). \tag{5}$$

Throughput is summarized from the recorded per-repetition throughput values rather than obtained by inverting the aggregated latency statistic. For the corresponding CPU and GPU executions, acceleration is calculated from the paired session-level timing medians:

$$S_s = \frac{\widetilde{T}_{\text{AES-NI},s}}{\widetilde{T}_{\text{GPU},s}}, \qquad \widetilde{S} = \text{median}_{s=1,\ldots,5}(S_s). \tag{6}$$

Here, $\widetilde{T}_{\text{AES-NI},s}$ and $\widetilde{T}_{\text{GPU},s}$ denote within-session median times for the same platform, mode, and input size. GPU resident and pipeline speedups are calculated separately against the corresponding AES-NI time. Values of $S_s > 1$ indicate a faster GPU path, while values below one indicate a faster CPU reference. The reported $\widetilde{S}$ is the median of five paired ratios, not a ratio of independently aggregated CPU and GPU throughput values.

The analysis reports session ranges, interquartile ranges, sample coefficients of variation, and counts of sessions with $S_s > 1$. For five ordered session estimates, the interquartile range is the fourth minus the second value; the sample coefficient of variation is the sample standard deviation divided by the mean. An advantage in every session is recorded as $5/5$ directional consistency. These are

descriptive statistics, not confidence intervals or formal significance tests. The twenty repetitions within a session are not treated as independent sessions, and measurements are not removed merely because they appear extreme.

The use of repeated sessions, paired workloads, and documented execution conditions follows recommendations on experimental bias and hierarchical variability in performance measurement [17–19]. Raw observations and their provenance are retained for verification of the session estimates and acceleration calculations.

3.2 Native CPU Baseline and Same-Runtime T4 Validation

The historical three-platform study used a hardware-assisted PyCryptodome host path. A later, independent T4 study compared that path, a native OpenSSL 3 EVP AES-128 implementation, and the original validated CUDA baseline on the same runtime allocation. The five sessions used seeds 2003–2007, five warm-ups and 20 measurements per size and mode; buffers ranged from 1 KiB to 32 MiB. Results from this newer study are paired only within its own sessions.

Native *CPU transform time* was measured inside C around encryption with prepared contexts/output; it excludes the Python-to-C call, allocation and cipher setup. GPU *resident time* is a CUDA-event compute interval with prepared device buffers. GPU *prepared pipeline time* includes payload host-to-device transfer, kernel execution, device-to-host transfer and synchronization, but excludes GPU allocation, staging, key expansion and output conversion. These boundaries are useful for determining whether prepared GPU processing repays transfer cost, but they do not provide identical wrapper costs or full application latency. CPU operation-envelope measurements include additional setup/allocation/copying and are retained only as asymmetric diagnostics.

3.3 Independent-Page CTR Batch Experiment

A separate T4 campaign encrypted $N \in \{1, 4, 16, 32\}$ independently managed 1 MiB pages, for total sizes of 1, 4, 16 and 32 MiB. Each page used a separate 96-bit synthetic test nonce concatenated with a 32-bit big-endian initial counter of one. The counter is advanced *within* each page and never carried into another page's nonce. The page-aware CUDA kernel assigns blocks to their corresponding page and encrypts all pages in one launch using the unchanged historical AES round implementation. A native OpenSSL EVP path processes the page contexts sequentially on one calling CPU thread.

An equal-size continuous CTR stream forms the control. For each size and repetition the page and continuous controls share plaintext and key but use distinct nonce domains. The experiment validates ciphertext independently against a CPU reference, including published vectors, page boundaries, counter carry and overflow guards. Five T4 sessions (seeds 2003–2007) each include five warm-ups

and 20 measurements for all four batch sizes, both layouts and three scheduled measurement paths. This yields 2,400 measured observations plus 600 warm-ups. Actual execution order, input digests, source hashes, software versions and session identity are retained. All reported speedups use the CPU duration divided by GPU duration: values greater than one favor the GPU. Five session medians, not pooled individual repetitions, form the primary descriptive replication units. The study is synthetic and measures batch completion rather than application request latency.

4 Results

Figure 1 presents median paired GPU speedups across the five input sizes, separately for AES-128 ECB and CTR and for resident and prepared-buffer execution. At 32 MiB, resident execution exceeds the corresponding AES-NI reference on all three systems, in both modes and in every session. Pipeline results differ by platform and mode: CTR gains occur in all five sessions on T600 and T4, whereas the remaining platform–mode combinations have mixed or consistently slower pipeline results.

Table 2 provides an overview of the 32 MiB findings, with resident and pipeline medians, pipeline session ranges, and directional counts. Bold pipeline values identify improvements observed in all five sessions. The table retains both modes and all three systems, including cases with a median below parity and cases whose session range crosses parity.

Table 2
Paired speedup over AES-NI at 32 MiB. R: resident; P: pipeline. Brackets show the five-session pipeline range; the last column counts P gains.

System	Mode	R	P	P range	$P > 1$
GTX	ECB	3.091	0.761	[0.722, 0.818]	0/5
GTX	CTR	3.693	0.935	[0.918, 1.008]	1/5
T600	ECB	2.215	1.118	[0.659, 1.144]	3/5
T600	CTR	2.666	**1.354**	[1.129, 1.391]	**5/5**
T4	ECB	2.207	0.893	[0.868, 0.901]	0/5
T4	CTR	11.758	**4.761**	[4.691, 5.021]	**5/5**

Bold pipeline medians and counts indicate acceleration in all five sessions. Resident execution is faster in all five sessions for every row.

For T600 CTR, the median pipeline speedup of 1.354× corresponds to approximately 26.14% lower latency, calculated from the median paired CPU-to-GPU timing ratio. Its session range remains above parity, from 1.129× to 1.391×. The paired speedup has an interquartile range of 0.030896 and a sample coefficient of variation of approximately 8.18%. These statistics describe the five session-level

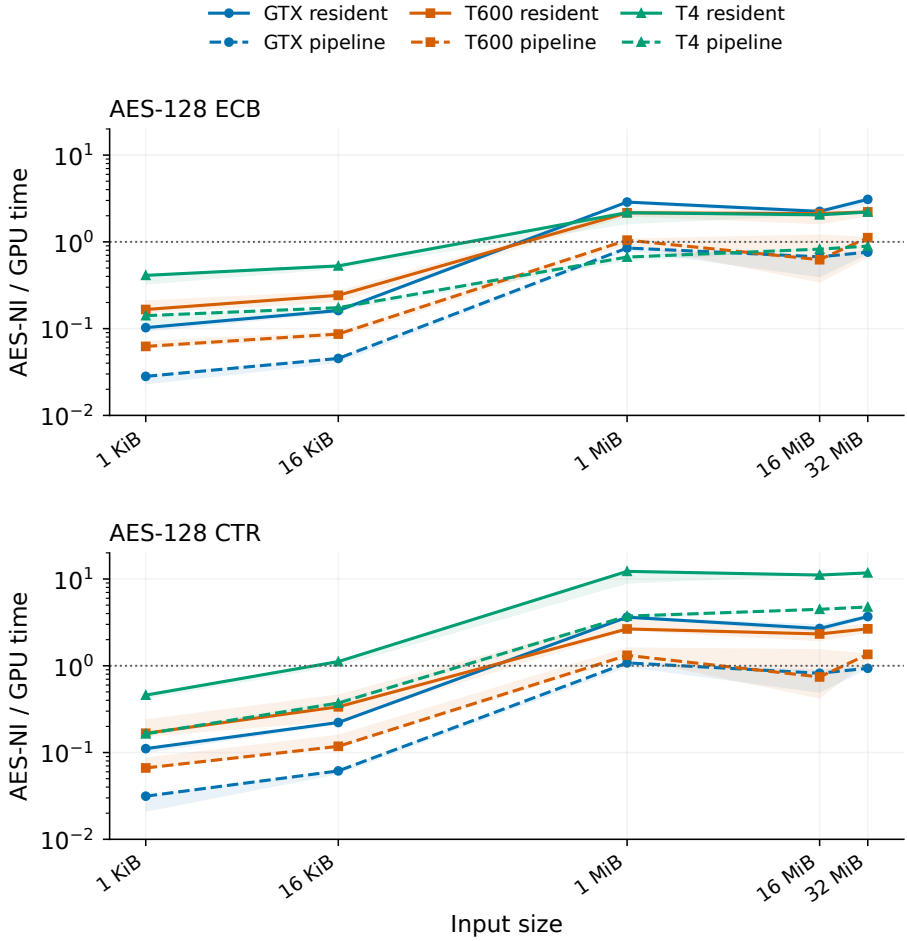


Fig. 1 — GPU-assisted AES-128 acceleration across input sizes and encryption modes. Curves represent median paired speedups relative to the corresponding host AES-NI implementation, with one panel per mode and distinct colors and markers for each GPU system. Solid lines denote resident execution; dashed lines denote the prepared-buffer pipeline. Resident execution measures device-side computation, whereas the prepared-buffer pipeline includes synchronous host–device transfers and synchronization. Shaded regions represent the observed ranges across five study sessions, not confidence intervals. The horizontal reference denotes parity.

ratios used for this configuration.

T4 CTR has the largest median pipeline speedup at 32 MiB, 4.761×, with a five-session range of 4.691–5.021×. For both T600 and T4 CTR, every recorded session favors the pipeline even though the measured interval includes transfers and synchronization. The resident speedups are larger than the pipeline speedups on both systems, as they are for every platform–mode combination in Table 2.

GTX CTR has a pipeline median of 0.935× and favors the GPU in one session. Its range, 0.918–1.008×, crosses parity. GTX ECB remains below parity in all five sessions. T600 ECB has a median above one, 1.118×, but only three sessions favor the GPU, with a range of 0.659–1.144×. T4 ECB remains slower than AES-NI throughout the five sessions despite resident speedups above one. Thus, the positive T600 ECB median coexists with mixed session outcomes, while the T600 and T4 CTR medians accompany exclusively positive outcomes.

Table 3 reports absolute GPU throughput at the same input size. T4 has the highest resident and pipeline CTR rates among the tested systems. GTX resident CTR throughput exceeds T600 resident throughput, while the ordering is reversed for the pipeline. On each system, the pipeline rate is below the corresponding resident rate in both encryption modes.

Table 3
GPU throughput at 32 MiB in decimal MB/s. Values are medians of five session-level throughput medians.

System	Mode	Resident	Pipeline
GTX	ECB	4797.6	1207.8
GTX	CTR	4585.6	1188.9
T600	ECB	4451.1	1997.8
T600	CTR	4328.9	2049.4
T4	ECB	9176.9	3713.0
T4	CTR	9166.8	3713.9

The T4 GPU rates are nearly identical across modes: approximately 3713 MB/s for ECB and 3714 MB/s for CTR in the pipeline, and 9177 and 9167 MB/s, respectively, in resident execution. In contrast, its host-relative pipeline speedups are 0.893× for ECB and 4.761× for CTR. The host AES-NI ECB/CTR throughput ratio has a session median of 5.378× and a range of 5.250–5.795×. The corresponding host ratios on GTX and T600 are approximately 1.26×. The T4 host therefore records higher ECB than CTR throughput in every session, while its measured GPU throughput changes little between modes. Figure 2 compares the absolute rates of all three execution paths, using the same vertical scale for both modes.

The smaller workloads in Figure 1 show different acceleration outcomes. At

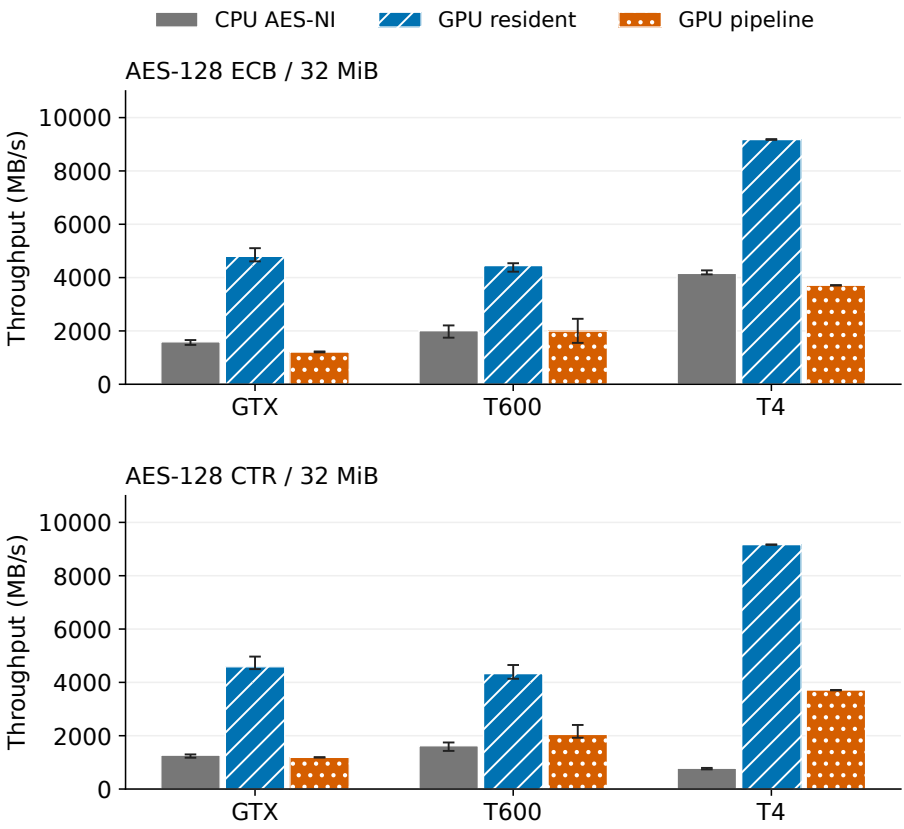


Fig. 2 — AES-128 throughput at 32 MiB for CPU AES-NI, resident GPU execution, and the prepared-buffer GPU pipeline. Bars show medians of five session-level throughput medians; error bars span the observed session minima and maxima, not confidence intervals. Throughput uses decimal MB/s. The GPU pipeline includes synchronous host–device transfers and synchronization; resident execution excludes these transfers. CPU bars represent the corresponding host implementation on each system.

1 KiB, both resident and pipeline paths are slower than AES-NI in every session, on all systems and in both modes. At 16 KiB, all pipeline configurations remain slower in every session. Resident T4 CTR has a median above parity at that size, with gains in four sessions; the other resident configurations remain below parity throughout. At 1, 16, and 32 MiB, resident execution is faster than AES-NI in all sessions for every platform and mode.

Table 4 summarizes CTR pipeline results across the size grid. T600 gains occur in all sessions at 1 and 32 MiB, but in only one session at 16 MiB. T4 gains occur in all sessions at each of the three MiB-scale inputs. GTX has four favorable sessions at 1 MiB, none at 16 MiB, and one at 32 MiB; no tested size has five favorable GTX pipeline sessions. ECB pipeline results likewise contain no five-session improvement on GTX or T4. T600 ECB favors the GPU in four, two, and three sessions at 1, 16, and 32 MiB, respectively.

Table 4
CTR pipeline speedup across input sizes. Each entry gives the median paired speedup and, in parentheses, the number of sessions faster than AES-NI out of five. Bold entries have gains in all five sessions.

Input	GTX	T600	T4
1 KiB	0.031 (0/5)	0.066 (0/5)	0.164 (0/5)
16 KiB	0.061 (0/5)	0.118 (0/5)	0.372 (0/5)
1 MiB	1.080 (4/5)	**1.320 (5/5)**	**3.734 (5/5)**
16 MiB	0.820 (0/5)	0.744 (1/5)	**4.478 (5/5)**
32 MiB	0.935 (1/5)	**1.354 (5/5)**	**4.761 (5/5)**

Absolute throughput is non-monotonic across the size grid. On GTX, throughput at 16 MiB is below both the 1 MiB and 32 MiB rates in all five sessions, for both modes and both GPU timing boundaries. On T4, this local reduction appears in resident throughput in four sessions for each mode, while neither pipeline mode shows the same pattern. The session counts therefore differ between platforms and between the two timing boundaries on T4.

4.1 Oracle Cloud ARM64: CPU Implementation and Envelope Controls

On the independent Oracle Cloud ARM64 host, the five-session 32 MiB AES-128-CTR study produced approximately 4.03 GB/s of native OpenSSL-default transform-only throughput, compared with approximately 209 MB/s for the PyCryptodome path and 218 MB/s for the `OPENSSL_armcap=0` control. These values describe the specific library implementations and timed boundaries; they are not GPU speedups. The default OpenSSL path was approximately 19.3 times faster than the recorded PyCryptodome transform path at this size, with the important

qualification that Python/FFI and C-native interval definitions differ.

In a separate host-side envelope experiment, a single worker achieved approximately 1.06 GB/s at 32 MiB, while two concurrent workers reached approximately 2.03 GB/s in aggregate, or about 1.92 times the single-worker aggregate rate. These campaigns are not paired per-repetition observations and their envelope boundary differs from the 4.03 GB/s native transform interval. The contrast demonstrates that buffer lifecycle and concurrency must be considered before extending cryptographic core throughput to a complete service. No Oracle result is used to calculate a speedup against the Tesla T4 or other GPU system.

4.2 Revisiting the Host Baseline on the Tesla T4

The historical 4.761× T4 CTR speedup in the preceding tables is an *earlier PyCryptodome-relative result*, not a speedup relative to an optimized native OpenSSL implementation. In a new five-session T4 campaign, 32 MiB AES-128-CTR yields a 5.586× median session ratio against PyCryptodome but only 0.942× against native OpenSSL when the GPU prepared pipeline is the denominator. The OpenSSL transform throughput is approximately 3.874 GB/s and the GPU prepared-pipeline throughput is approximately 3.646 GB/s. When only GPU resident computation is considered, the native OpenSSL CPU/GPU ratio is 2.299×. Thus the improved CPU baseline reverses the prepared-pipeline ranking, without implying that the historical measurements were incorrect.

4.3 Independent Pages: A Compute Advantage That Does Not Survive Transfers

Table 5 gives the prespecified native OpenSSL transform to prepared CUDA pipeline time ratios in the page-batching study. Every reported ratio is below unity across all five sessions. The gap narrows at larger batch sizes but does not reverse direction.

Table 5
Tesla T4 Phase 2: median of five session-level CPU OpenSSL transform / GPU prepared-pipeline ratios. Values below one favor the CPU at this boundary.

Total payload	Continuous CTR	Independent 1 MiB pages
1 MiB	0.5437	0.5404
4 MiB	0.6948	0.7137
16 MiB	0.8163	0.8240
32 MiB	0.8963	0.9125

The most informative case is a batch of 32 independent pages. The median CPU native transform duration is approximately 8.362 ms, compared with 3.714 ms

for one GPU-resident kernel and 9.153 ms for the prepared GPU pipeline. Table 6 separates the measurements. The compute-only ratio is 2.251× in favor of the GPU, whereas the transfer-inclusive ratio is 0.913× and favors the CPU. Both directions are consistent across all five sessions. The observed H2D-plus-D2H component is about 5.418 ms. Medians of nested or separately timed components cannot be added as if they described a single observation.

Table 6
Tesla T4 Phase 2, 32 independent 1 MiB pages: separately summarized timings (five session medians).

Measurement boundary	Approximate time (ms)
Native OpenSSL CPU transform	8.362
CUDA resident kernel	3.714
GPU H2D + D2H transfers	5.418
CUDA prepared pipeline, including transfers and synchronization	9.153
GPU compute-only speedup versus CPU transform	2.251×
GPU prepared-pipeline speedup versus CPU transform	0.913×

The page-aware GPU prepared pipeline shows almost no relative penalty against a single continuous CTR stream of the same size: its page/continuous throughput ratios are approximately 1.0007× at 4 MiB, 1.0068× at 16 MiB and 1.0066× at 32 MiB. These sub-percent differences should not be described as a robust optimization gain. They support the narrower observation that independent per-page counter handling does not impose a large measured GPU pipeline overhead in the evaluated implementation. This is distinct from demonstrating an advantage from batching over sequential per-page GPU launches, which was not the control experiment.

5 Discussion

5.1 The Specific Favorable Case: Data Already on the GPU

The positive result is precise: for 32 independently encrypted 1 MiB pages, one CUDA resident AES-128-CTR launch takes approximately 3.714 ms, whereas native OpenSSL EVP requires approximately 8.362 ms for the prepared sequential CPU transformation. The measured 2.251× ratio makes GPU execution a credible compute-stage candidate when plaintext is already GPU-resident and the encrypted result is needed on the device for another processing stage. This is not a claim that an ordinary host-memory encryption service becomes 2.251× faster: the resident interval excludes data placement, GPU allocation and preparation, and a complete

multi-stage application has not been benchmarked.

Potential workload classes include GPU-native analytics pipelines, pipelines that prepare columnar buffers on the GPU before encryption, and device-side processing that passes the resulting ciphertext to a later GPU operation. The common factor is *avoided payload round trips*, not the mere presence of a GPU. Such placement can be attractive only if input data originate on the device or would need to move to it for other work anyway, and if the required output does not have to return immediately to the CPU. Where plaintext starts in host RAM and ciphertext is consumed by a host process, the tested prepared pipeline takes approximately 9.153 ms versus 8.362 ms for CPU transformation: the CPU is faster at these declared boundaries.

5.2 A Measured Data-Movement Budget, Not an Application Speedup

The 32 MiB prepared GPU pipeline is approximately 0.791 ms slower than the prepared CPU transformation, based on the displayed session-level median durations. This difference describes the margin to parity for the measured configuration, not an empirically achieved optimization. The approximately 5.418 ms transfer component demonstrates that data movement is large relative to AES kernel execution; its separately reported median is not directly substitutable into a single-iteration latency equation. Avoiding transfers could make GPU-resident processing competitive, but the exact crossover requires a new workload-level experiment including data preparation, input residency, output placement, any subsequent consumer, and buffer lifetime.

The ratio of CPU operation-envelope to GPU prepared-pipeline time is not a symmetrical end-to-end comparison: CPU allocation and setup are included, whereas GPU allocation, key upload, pinned-memory staging and final conversion are excluded. It must not be used to claim an application-level speedup. A second caution is that native OpenSSL EVP timings exclude the Python wrapper, unlike parts of the GPU path. The present study therefore provides a well-defined prepared-workload comparison rather than a common application API benchmark.

5.3 Independent Pages and Practical Storage Workloads

Correct independent CTR processing is more than slicing a continuous stream: each logical page needs its own nonce/counter domain, and the counter must not cross the domain boundary. The page-aware CUDA implementation verifies these conditions and processes a batch in one launch. At 32 MiB, the measured GPU pipeline throughput differs by less than 1% from the continuous control; hence handling page boundaries does not impose a large penalty in this specific kernel. The observation does not imply that page batching is faster than separately launching the pages: no separate-per-page launch baseline was included, and the

CPU processes the pages sequentially.

The Parquet `AES_GCM_CTR_V1` design [23] motivates the page geometry, but the experiment is not a Parquet writer, reader or encryptor. Format metadata, AAD, authentication, key management, compression and nonce-generation rules remain absent. Raw CTR encryption is malleable and provides no integrity protection. Practical storage deployment therefore requires a complete format-compliant and authenticated workflow; AES-CTR throughput alone cannot support security or end-to-end speedup claims about such a system.

5.4 Why the CPU Baseline Matters

The historical 4.761× T4 result used a PyCryptodome-based host interval. The fresh T4 campaign measured 5.586× against PyCryptodome but only 0.942× against native OpenSSL at the prepared GPU boundary. The native implementation makes the CPU substantially more competitive. Because library versions, API overhead and timing boundaries differ, this is evidence of baseline sensitivity rather than a proof that the old measurement was invalid. Historical GTX 1650 Ti and T600 findings remain useful for showing diversity across complete configurations but cannot substitute for a same-runtime OpenSSL-controlled study on those machines.

5.5 Cross-Architecture Evidence from Oracle ARM64

The Oracle CPU-only control strengthens the baseline-sensitivity argument: on the same ARM64 machine, native OpenSSL-default AES-128-CTR and the recorded PyCryptodome or capability-masked paths differ markedly. This does not show that ARM is intrinsically faster than x86, nor that all of the observed gap is due to AES instructions: implementations, Python interfaces, capability masking, and measurement scopes also matter. The two-worker envelope result additionally illustrates that aggregate throughput and single-request latency answer different questions. Thus, the portable finding is a methodological and architectural constraint: report an optimized, capability-aware host reference and the actual operation boundary before attributing an advantage to accelerator hardware. A future controlled ARM study should trace executed cryptographic instructions, harmonize native call boundaries, and examine additional core counts and AES-256.

5.6 Generalizability, Constraints and Future Work

The relevant engineering decision is whether a specific pipeline can retain plaintext and ciphertext on the GPU while maintaining its functional and security requirements. The following hypotheses are actionable but untested: (1) a GPU-first columnar transformation followed by resident encryption could avoid round-trip payload transfers; (2) encryption before another GPU-resident consumer could

make cipher output placement beneficial; and (3) pipelined transfers or batch reuse might reduce the effective movement cost for host-resident data. None is demonstrated by the present one-launch benchmarks.

The next experiment should implement an actual producer–encrypt–consumer CUDA pipeline, with device-resident buffers, an explicit encrypted-output consumer, correctness validation, and a CPU counterpart completing the same functional work. It should account for input acquisition, context/key and nonce-table setup, GPU allocation, transfers actually needed, synchronization and output disposition. A separate one-page-at-a-time GPU launch baseline would isolate the genuine *batching* effect from the present continuous CTR control. Format-level work would integrate Parquet's full encryption rules, authenticated metadata, and key management, then report file or query throughput and latency. AES-GCM, AES-256, overlapping streams and multiple simultaneous requests are further experiments, not consequences of AES-128-CTR results.

The present measurements use one CUDA implementation and a single-threaded native CPU comparator, five sessions, synthetic data and a shared cloud platform. Kernel-level and transfer timings use different instrumentation; GPU scheduling, memory placement, host contention and CPU dispatch affect results. No instruction-level trace was captured. The work neither ranks all available CPUs/GPUs nor estimates a universal cross-platform crossover.

6 Conclusion

The experiments identify a specific opportunity rather than a blanket GPU advantage: with 32 independent 1 MiB pages, Tesla T4 AES-128-CTR resident execution is approximately 2.251× faster than native OpenSSL EVP transformation at the measured compute boundaries. Once the prepared GPU path includes synchronous host–device transfers and synchronization, that ratio falls to 0.913× and the CPU wins in all five sessions. A page-aware kernel with independent counter domains has approximately the same GPU pipeline throughput as a continuous stream of equal size, but this does not establish an acceleration benefit over separate per-page launches.

A second key finding is baseline sensitivity: the new T4 study reports a 5.586× prepared-pipeline ratio relative to PyCryptodome but 0.942× relative to native OpenSSL. Consequently, library choice and measurement boundaries are essential to any defensible GPU acceleration claim. Earlier three-platform measurements remain historical evidence tied to their original host implementation, not proof of an optimized-CPU advantage.

An independent Oracle Cloud ARM64 CPU-only control further establishes implementation sensitivity outside x86: at 32 MiB AES-128-CTR, native OpenSSL-

default transform throughput was approximately 4.03 GB/s, versus 209 MB/s for the recorded PyCryptodome path, while aggregate operation-envelope throughput increased from approximately 1.06 GB/s with one worker to 2.03 GB/s with two. These differently bounded observations are not compared directly with GPU measurements.

The practical candidate is an application that already keeps its input and subsequent ciphertext consumer on the GPU; in such a pipeline encryption can potentially exploit device-resident computation without paying a full payload round trip. This is an engineering hypothesis supported by the isolated resident result, not an established application speedup. Future work should measure a complete GPU-native producer–encrypt–consumer pipeline against an equivalent CPU workload and separately test one-page-per-launch overhead, Parquet-compliant authenticated storage processing, and overlapping data movement. The principal finding is therefore a reproducible boundary between GPU computational gain and transfer-limited execution, together with a validated independent-page workload for investigating that boundary.

References

[1] National Institute of Standards and Technology. Advanced encryption standard (AES). Federal Information Processing Standards Publication 197, Update 1, National Institute of Standards and Technology, 2023. URL `https://doi.org/10.6028/NIST.FIPS.197-upd1`.

[2] Morris Dworkin. Recommendation for block cipher modes of operation: Methods and techniques. NIST Special Publication 800-38A, National Institute of Standards and Technology, 2001. URL `https://doi.org/10.6028/NIST.SP.800-38A`.

[3] Svetlin A. Manavski. CUDA compatible GPU as an efficient hardware accelerator for AES cryptography. In *Proceedings of the IEEE International Conference on Signal Processing and Communications*, pages 65–68, 2007. doi: 10.1109/ICSPC.2007.4728256.

[4] Owen Harrison and John Waldron. AES encryption and decryption on the GPU. In *GPU Gems 3*, chapter 36. Addison-Wesley Professional, 2007. URL `https://developer.nvidia.com/gpugems/gpugems3/part-vi-gpu-computing/chapter-36-aes-encryption-and-decryption-gpu`.

[5] Keisuke Iwai, Naoki Nishikawa, and Takakazu Kurokawa. Acceleration of AES encryption on CUDA GPU. *International Journal of Networking and Computing*, 2(1):131–145, 2012. doi: 10.15803/ijnc.2.1_131.

[6] Canhui Wang and Xiaowen Chu. GPU accelerated AES algorithm, 2019. URL `https://arxiv.org/abs/1902.05234`.

[7] Cihangir Tezcan. Optimization of advanced encryption standard on graphics processing units. *IEEE Access*, 9:67315–67326, 2021. doi: 10.1109/ACCESS.2021.3077551.

[8] Wai-Kong Lee, Seog Chung Seo, Hwajeong Seo, Dong Cheon Kim, and Seong Oun Hwang. Speed record of AES-CTR and AES-ECB bit-sliced implementation on GPUs. *IEEE Embedded Systems Letters*, 16(4):481–484, 2024. doi: 10.1109/LES.2024.3409725.

[9] Intel Corporation. Intel advanced encryption standard instructions (AES-NI), 2012. URL

https://www.intel.com/content/www/us/en/developer/articles/technical/advanced-encryption-standard-instructions-aes-ni.html.

[10] Victor W. Lee, Changkyu Kim, Jatin Chhugani, Michael Deisher, Daehyun Kim, Anthony D. Nguyen, Nadathur Satish, Mikhail Smelyanskiy, Srinivas Chennupaty, Per Hammarlund, Ronak Singhal, and Pradeep Dubey. Debunking the 100X GPU vs. CPU myth: An evaluation of throughput computing on CPU and GPU. In *Proceedings of the 37th Annual International Symposium on Computer Architecture*, pages 451–460, 2010. doi: 10.1145/1815961.1816021.

[11] Chris Gregg and Kim Hazelwood. Where is the data? why you cannot debate CPU vs. GPU performance without the answer. In *2011 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 134–144, 2011. doi: 10.1109/ISPASS.2011.5762730.

[12] NVIDIA Corporation. CUDA C++ best practices guide, n.d. URL https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/. Accessed 2026.

[13] Mark Harris. How to overlap data transfers in CUDA C/C++. NVIDIA Technical Blog, 2012. URL https://developer.nvidia.com/blog/how-overlap-data-transfers-cuda-cc/.

[14] Morris Dworkin. Recommendation for block cipher modes of operation: Galois/counter mode (GCM) and GMAC. NIST Special Publication 800-38D, National Institute of Standards and Technology, 2007. URL https://doi.org/10.6028/NIST.SP.800-38D.

[15] Georg Schönberger and Jürgen Fuß. GPU-assisted AES encryption using GCM. In *Communications and Multimedia Security*, volume 7025 of *Lecture Notes in Computer Science*, pages 178–185. Springer, 2011. doi: 10.1007/978-3-642-24712-5_16.

[16] JaeSeok Lee, DongCheon Kim, and Seog Chung Seo. Parallel implementation of GCM on GPUs. *ICT Express*, 11(2):310–316, 2025. doi: 10.1016/j.icte.2025.01.006.

[17] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 265–276, 2009. doi: 10.1145/1508244.1508275.

[18] Tomas Kalibera and Richard E. Jones. Rigorous benchmarking in reasonable time. In *Proceedings of the International Symposium on Memory Management*, pages 63–74, 2013. doi: 10.1145/2464157.2464160.

[19] Torsten Hoefler and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 73:1–73:12, 2015. doi: 10.1145/2807591.2807644.

[20] PyCryptodome. PyCryptodome documentation, n.d. URL https://www.pycryptodome.org/. Accessed 2026.

[21] CuPy. cupy.RawKernel, n.d.. URL https://docs.cupy.dev/en/stable/reference/generated/cupy.RawKernel.html. Accessed 2026.

[22] CuPy. Performance best practices, n.d.. URL https://docs.cupy.dev/en/stable/user_guide/performance.html. Accessed 2026.

[23] Apache Parquet Project. Parquet Modular Encryption. Apache Parquet file format documentation. URL https://parquet.apache.org/docs/file-format/data-pages/encryption/. Accessed: 2026-10-27.